

Kubernetes and Cloud Native Associate - KCNA

Scheduling

Taints and Tolerations

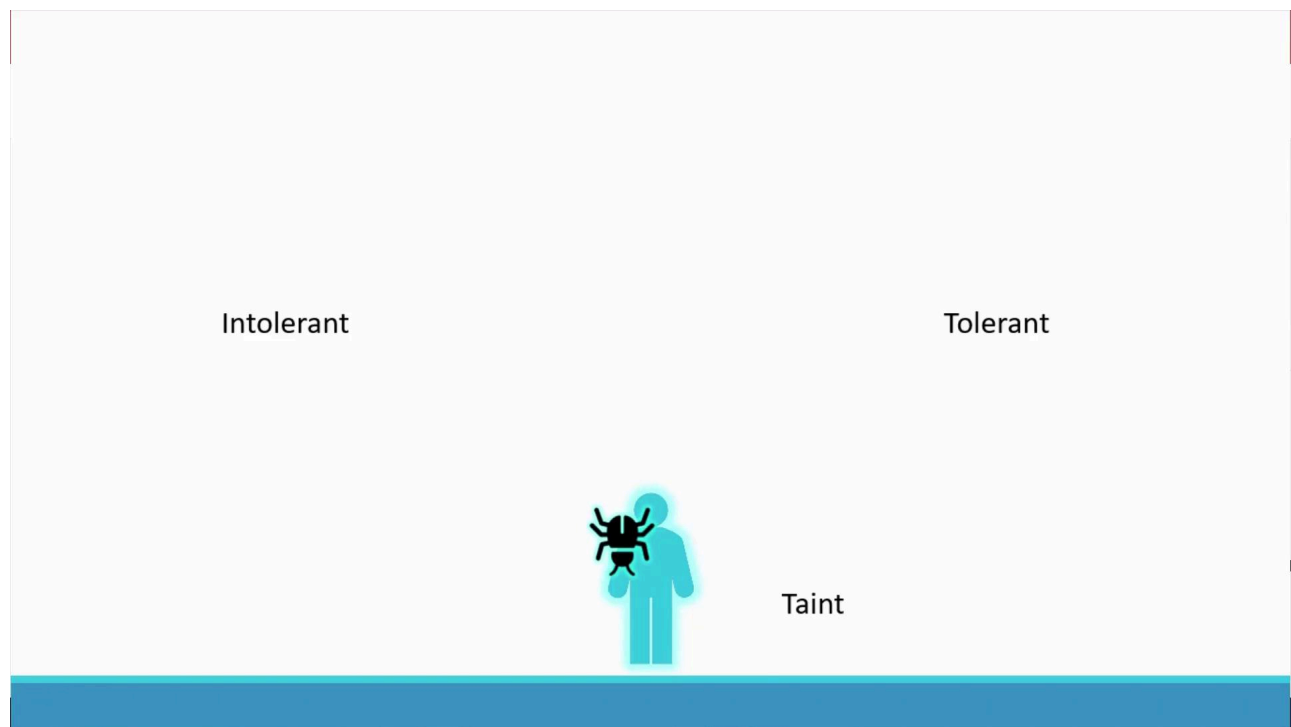
In this article, we explore how Kubernetes uses taints and tolerations to control the scheduling of pods onto nodes. We will explain the concepts using a clear analogy and provide step-by-step examples to help you understand how to restrict pod placements effectively.

Understanding the Analogy

Imagine a bug approaching a person. To prevent the bug from landing, you spray the person with a repellent. In this analogy:

- The repellent represents a **taint** applied to a node.
- The bug corresponds to a **pod** deciding whether to land.
- A bug's sensitivity or resistance to the repellent represents a **toleration**.

Pods without the appropriate toleration will be repelled by taints, while those with a matching toleration can be scheduled on tainted nodes.



Translating the Analogy to Kubernetes

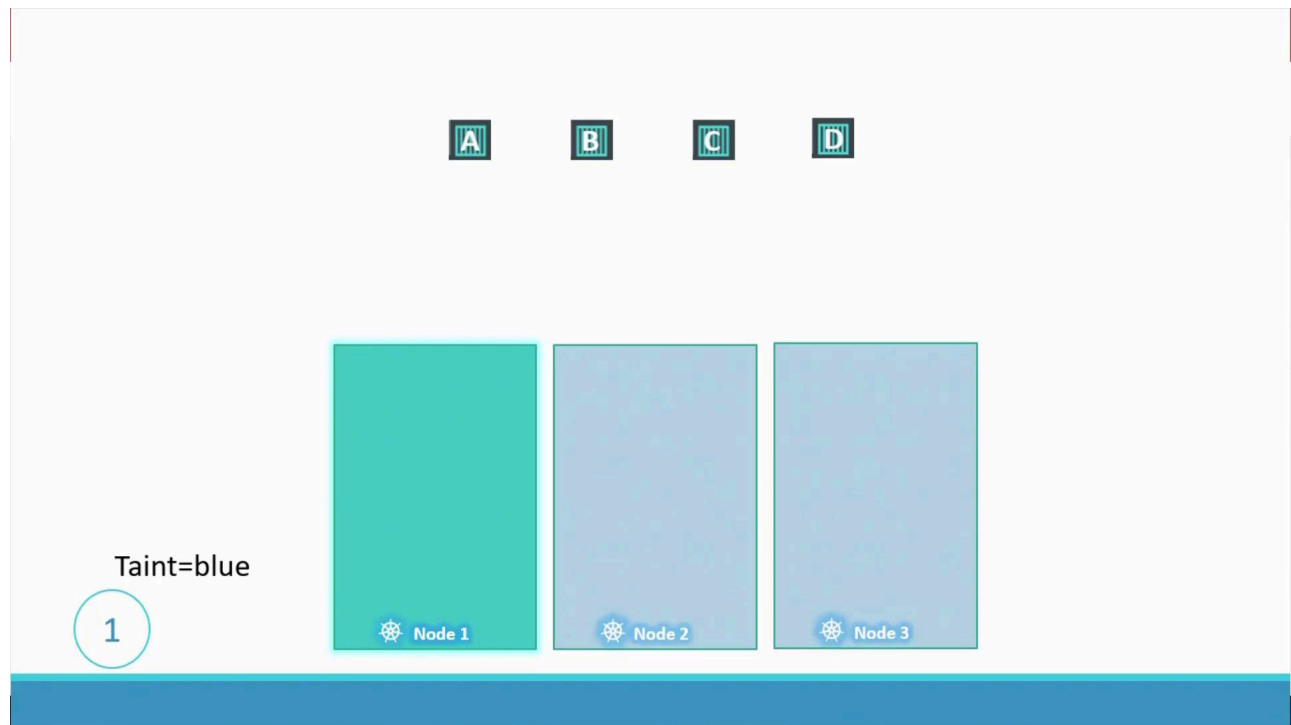
In Kubernetes:

- **Nodes** act as the "person" that is tainted.
- **Pods** act as the "bugs" that may be scheduled based on their tolerations.
- **Taints** on nodes repel pods that do not have matching tolerations.
- **Tolerations** in pods allow them to overcome the taint and be scheduled on that node.

Note that taints and tolerations are solely used for scheduling restrictions, not cluster security.

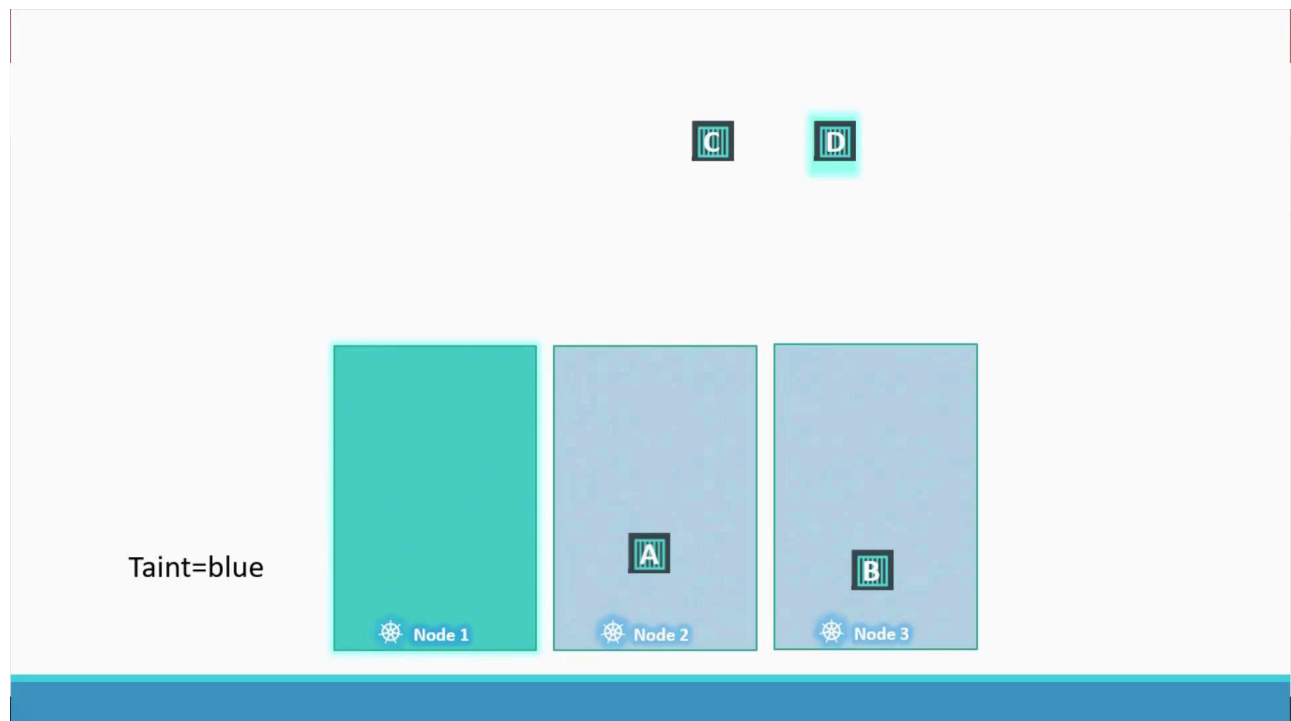
Example Scenario

Consider a cluster with three worker nodes (named one, two, and three) and four pods (labeled A, B, C, and D). Initially, the scheduler distributes the pods uniformly across the nodes. To dedicate node one for a particular application, a taint (for example, "blue") is applied to node one. With the taint in place, pods without any toleration will not be scheduled on node one.



To allow only a specific pod (pod D) to run on node one, a toleration for the taint is added to its configuration. The scheduler then proceeds as follows:

- Pods A, B, and C are repelled from node one because they do not tolerate the taint and are scheduled on nodes two and three.
- Pod D, with the matching toleration, is scheduled on node one.



Applying Taints on Nodes

You can taint a node using the following command:

```
kubectl taint nodes node-name key=value:taint-effect
```

For example, to taint node1 with `app=blue` and prevent pods from being scheduled on it by default (using the NoSchedule effect), run:

```
kubectl taint nodes node1 app=blue:NoSchedule
```



Taint Effects

There are three taint effects available:

- **NoSchedule:** Pods without the toleration are not scheduled on the node.
- **PreferNoSchedule:** The scheduler avoids placing pods on the node, but placement is not strictly prevented.
- **NoExecute:** New pods are not scheduled, and existing pods without tolerations are evicted.

Configuring Tolerations in Pods

To allow a pod to run on a tainted node, you must include a toleration in the pod's YAML definition. Below is an example configuration for a pod that tolerates the `app=blue` taint with the `NoSchedule` effect:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "blue"
      effect: "NoSchedule"
```

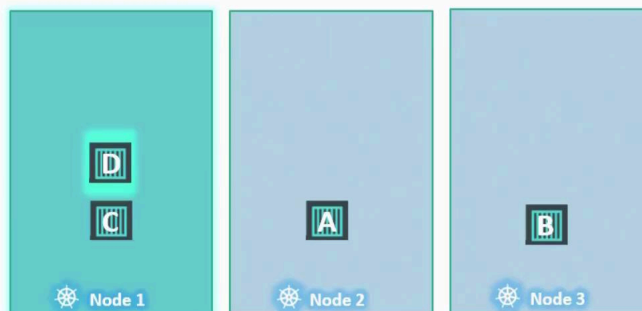
When this pod is created or updated, it can be scheduled on nodes with the matching taint while other pods without the toleration will be placed on different nodes.

Special Case: NoExecute Effect

Let's consider the NoExecute taint effect in a dedicated node scenario:

- Imagine three nodes with evenly distributed workloads.
- Node one is tainted with NoExecute, and only pod D has a toleration for this taint.
- As a result, any pod on node one without the required toleration, such as pod C, will be evicted, leaving only pod D running on node one.

Taint - NoExecute



Important Considerations

⚠ Scheduling vs. Placement

Remember that taints and tolerations only control which nodes can accept pods. They do not force a pod to run on a specific node. If node one is not selected by the scheduler, even a pod with the correct toleration might be placed elsewhere. For targeted node selection, consider using node affinity.

Moreover, while master nodes are fully capable of running pods, they are usually tainted by default to prevent non-management workloads from being scheduled on them. To view the master node taint, run:

```
kubectl describe node <master-node-name>
```

This command displays the taint information that ensures the stability and performance of cluster management services.

Conclusion

In this guide, we have:

- Explained how taints and tolerations work in Kubernetes using a relatable analogy.
- Demonstrated how to taint nodes and configure tolerations in pod specifications.
- Highlighted the differences between the NoSchedule, PreferNoSchedule, and NoExecute taint effects.
- Discussed best practices for controlling pod placement while noting the additional considerations for using node affinity.

By understanding taints and tolerations, you can effectively manage pod scheduling and optimize your cluster's performance and reliability.

For further reading:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)

This concludes our discussion on taints and tolerations in Kubernetes. Enjoy exploring and optimizing your Kubernetes clusters!



Watch Video

Watch video content

Labels and Selectors

Node Selectors



Beta



Search docs...



Kubernetes and Cloud Native Associate - KCNA

